Критический путь: за рамками базового расчёта

Что мы уже знаем (занятие 5)

- ES/EF/LS/LF, Total Float и Free Float
- 6-шаговый алгоритм расчёта критического пути
- Типичные ошибки, включая циклическую зависимость
- Краткий принцип сжатия сроков (crashing)

Что добавляет это занятие

- Альтернативные способы представить зависимости работ (не только сеть)
- Полный расчёт стоимости сжатия сроков, а не только принцип
- Научное обоснование правил построения WBS
- Практика итеративного уточнения расчёта
- Сильные и слабые стороны метода — включая то, чего CPM не умеет

Сеть — не единственный способ представить зависимости

Сетевой метод

то, что мы уже делали

CPM, PERT — работа как узел или дуга графа, зависимости — стрелки. Хорошо показывает последовательность и позволяет считать критический путь.

Матричный метод (DSM)

Design Structure Matrix

работы — строки и столбцы одной таблицы, зависимости — отметки вне диагонали. Хорошо показывает работы с обратной связью и большим числом связей — то, что на сети выглядело бы запутанным клубком стрелок.

Системная динамика

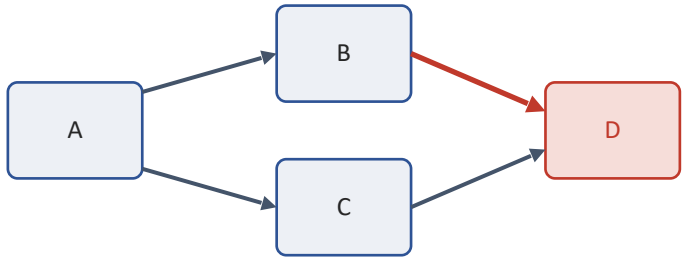
потоки и петли обратной связи

работа представлена не точкой на графике, а «запасом» (сколько ещё осталось сделать) и «потоком» (с какой скоростью делается) — подходит для работ с постоянными переделками, а не разовым проходом.

Сегодня подробно разберём матричный метод (DSM) — он лучше сети показывает то, что легко потерять в сложном сетевом графике.

Как эти три метода выглядят и где реально применяются

Сетевой метод (CPM/PERT)



Где применяется: проекты с преимущественно однонаправленными зависимостями и жёстким сроком — строительство, запуск IT-продукта (наши командные проекты), контрактные работы с фиксированной датой сдачи.

Как используется: строится сеть → считается критический путь → на нём фокусируется контроль сроков.

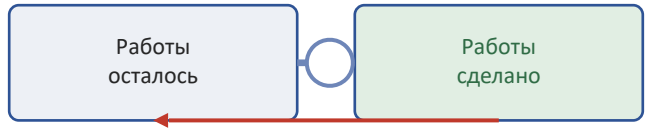
Матричный метод (DSM)

	A	B	C	D
A	—			
B	X	—		
C	X	X	—	
D		X	X	—

Где применяется: сложные инженерные системы с множеством взаимозависимых подсистем — например, совместные аэрокосмические проекты нескольких организаций.

Как используется: находят подсистемы с наибольшим числом связей — именно их интеграцию планируют и тестируют в первую очередь.

Системная динамика



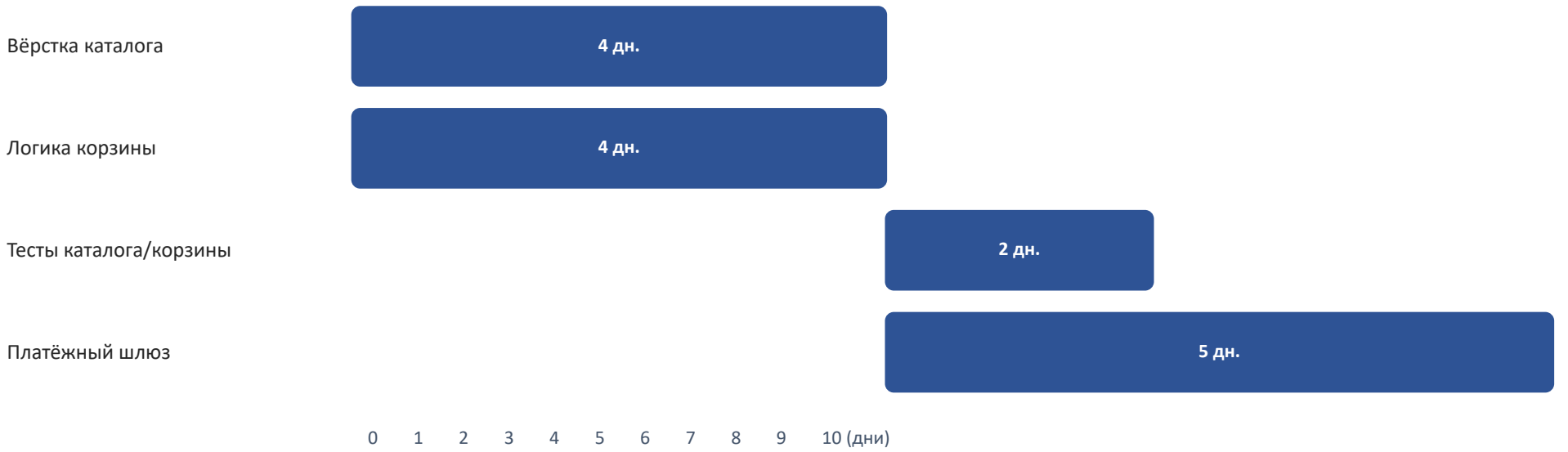
часть работы возвращается на переделку

Где применяется: проекты с высокой долей переделок — разработка ПО с непрерывным циклом тестирования и багфиксов, R&D с частыми итерациями, где часть «готовой» работы регулярно возвращается в переработку.

Как используется: моделируют не отдельные работы, а темпы потоков — сколько «перетекает» из «осталось» в «сделано» за период, с учётом возврата на переделку.

Диаграмма Гантта: удобство ценой слепой зоны

Названа в честь Генри Гантта (1861–1919); использовалась ещё для планирования постройки грузовых судов в Первую мировую войну. Сегодня это самый узнаваемый инструмент планирования — но у него есть системное ограничение.



⚠ Главное ограничение

Диаграмма Гантта не показывает явно, ПОЧЕМУ работы идут именно в таком порядке — зависимости приходится держать в голове или искать отдельно. Именно поэтому черновик расписания стоит сначала строить как сетевой график (где зависимости — это сами стрелки), и только потом переносить в Гантт для наглядности и отслеживания хода работ.

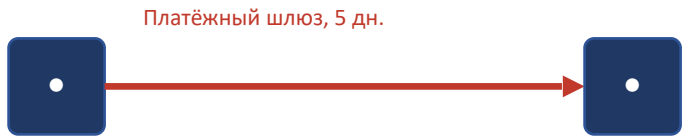
Работа как узел или как стрелка: два способа рисовать сеть

Работа = узел (то, чем мы пользовались)



Precedence Diagramming Method (PDM). Узлы — работы, стрелки — просто зависимости («после этого — то»). Стандарт в PMBOK, MS Project и на всех наших схемах.

Работа = стрелка (историческая альтернатива)



Kelley-Walker form. Сама работа — стрелка, а узлы — «состояния проекта» (точки во времени). Исторически более ранняя нотация: именно в этом виде метод придумывался в DuPont, но требует «фиктивных» связей для сложных зависимостей.

Почему сегодня везде используется первый способ: он проще читается, не требует «фиктивных» работ для условий вроде «С зависит и от А, и от В» (то, что мы явно видели на схеме ФлораЭкспресс), и однозначно ложится на табличный формат Excel.

Сколько уровней и веток должно быть в WBS? Не произвольно

Не более 5–9 работ на одном уровне декомпозиции

человеческая кратковременная память держит одновременно 7±2 единицы информации (классический эффект Миллера, 1956) — больше веток на одном уровне не улучшает планирование, а перегружает внимание

Не более 3–4 уровней вложенности

глубже — WBS теряет читаемость и начинает требовать отдельных подпроектов для управления

Не более 100–200 терминальных элементов на весь проект

если больше — это сигнал разбить не на работы, а на подпроекты со своими WBS

Работы одного уровня — сопоставимого размера

если один Work Package занимает 2 дня, а соседний на этом же уровне — 40, скорее всего, декомпозиция проведена неровно

Проверьте свой WBS с занятия 4 по этим правилам — если у одного эпика 12 работ, а у другого 2, это повод пересмотреть глубину декомпозиции, а не только проверять сумму процентов.

Расчёт критического пути — это не разовое действие

1. Первый черновой расчёт

берутся грубые, экспертные оценки длительности — и строится первая версия сети и критического пути

2. Переоценка критических и «почти критических» работ

именно эти работы уточняются в первую очередь — с ними связан наибольший риск для срока проекта; работы с большим резервом можно пока не трогать

3. Повтор расчёта

уточнённые оценки могут сдвинуть сам критический путь — переоценка снова запускает пересчёт

4. Остановка, когда критический путь стабилизировался

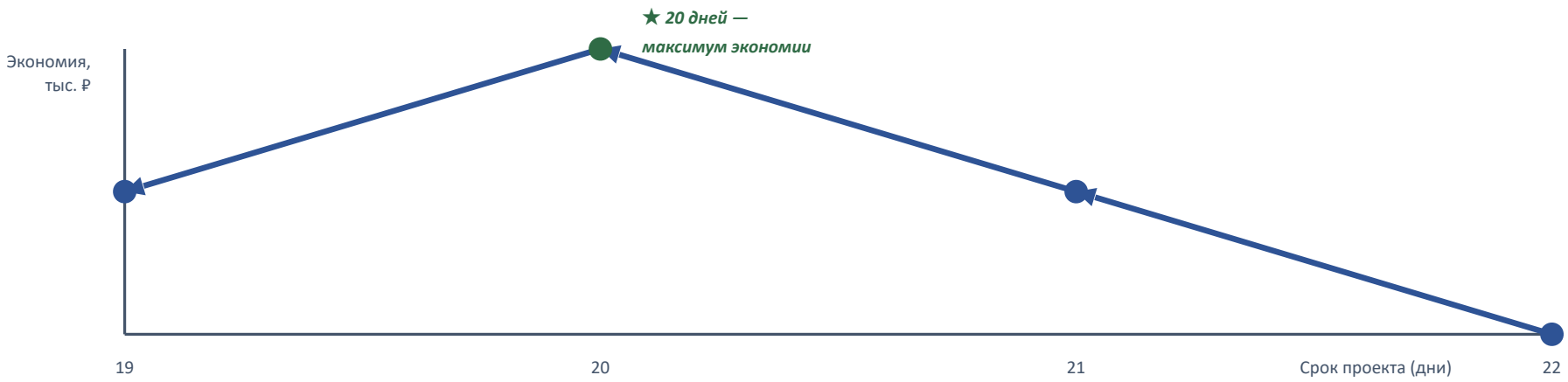
то есть очередная переоценка больше не меняет состав критических работ — уточнять дальше не имеет смысла

Практический смысл: не нужно тратить одинаковые усилия на уточнение всех 11 работ сети — сосредоточьтесь на тех, у кого Float = 0 или близко к нулю.

Сжатие сроков: конкретный расчёт для «ФлораЭкспресс»

Две критические работы можно ускорить за деньги: «Платёжный шлюз» — с 5 до 3 дней за 15 000 ₽/день; «Обработка ошибок оплаты» — с 3 до 2 дней за 25 000 ₽/день. Накладные расходы проекта — 20 000 ₽/день. Сжимаем сначала более дешёвую работу.

Срок проекта	Что сжато	Прямые затраты (нараст.)	Экономия на накладных	Чистый эффект
22 дня	—	0 ₽	0 ₽	база
21 день	Платёжный шлюз −1 дн.	15 000 ₽	20 000 ₽	+5 000 ₽ экономии
20 дней	**Платёжный шлюз −2 дн.**	**30 000 ₽**	**40 000 ₽**	**+10 000 ₽ экономии ★**
19 дней	+ Обработка ошибок −1 дн.	55 000 ₽	60 000 ₽	+5 000 ₽ экономии



Вывод: сжатие выгодно до 20 дней (зелёная точка — максимум чистой экономии, 10 000 ₽). Дальнейшее сжатие — уже вторая, более дорогая работа — снова начинает съедать выгоду: на 19 днях чистая экономия падает вдвое, до 5 000 ₽.

DSM: та же сеть, но в виде таблицы

Design Structure Matrix — работы одновременно и строки, и столбцы. Метка на пересечении строки i и столбца j означает: «работа i зависит от работы j». Возьмём знакомую структуру команд (A–F).

столбец = «от кого зависит»

строка = «кто зависит»

	A	B	C	D	E	F
A	—					
B		—				
C	X	X	—			
D		X		—		
E				X	—	
F			X		X	—

Читаем строку F: «X» стоит в столбцах C и E — значит, F зависит от C и от E. Столбец с несколькими «X» (как B) сразу показывает работу, от которой зависит многое, — на сложной сети с 50+ работами это увидеть по стрелкам значительно труднее, чем по одному переполненному столбцу таблицы.

У критического пути тоже есть слепые зоны

+ Сильные стороны

- Фокусирует внимание на узком подмножестве действительно важных работ
- Позволяет заранее оценить эффект от ускорения или замедления конкретной работы
- Даёт основу для расчёта экономики сжатия сроков

– Ограничения

- Не умеет работать с повторами и переделками (итерациями) — модель строго однонаправленная, «цикл» считается ошибкой, а не нормальным сценарием (в отличие от гибких методологий, где переделка — это часть процесса)
- Длительность работ считается детерминированной (одно число) — реальная неопределённость учитывается только через отдельный метод PERT, а не встроена в сам CPM
- Не подсказывает, что делать, если оценки просто неверны или предшественники перепутаны, — метод одинаково честно посчитает и правильную, и ошибочно заданную сеть

Почему у классического PM репутация «бюрократии с чек-листами»?

PM = Project Management, управление проектами

- не признаёт, что в реальных проектах бывают итерации и переделки — модель строится так, будто каждая работа выполняется ровно один раз
- негибкая: «изменение плана» часто воспринимается как провал менеджера, а не нормальная часть работы над проектом
- не мыслит вероятностно — расписание выглядит как единственное точное число, а не диапазон возможных исходов
- «заложник» существующего софта — планирование подстраивается под то, что умеет программа, а не наоборот
- работает в реактивном режиме — реагирует на отклонения постфактум, а не предупреждает о них заранее

Вопрос для обсуждения: какие из этих пяти претензий снимает Agile-подход, а какие остаются справедливыми даже для гибких команд?

Практика: примените новый материал к своей сети

1. DSM своей команды

постройте матрицу DSM для сети своего проекта (та же таблица, что и на слайде про DSM) — сравните, что заметнее на матрице, а что на сетевом графике

2. Проверка WBS

сверьте свой WBS с занятия 4 по 4 правилам со слайда 5 — найдите хотя бы один уровень, который стоило бы перестроить

3. Один раунд уточнения

выберите 2 критические и 2 близкие к критическим работы в своей сети и переоцените их длительность — обсудите, сдвинулся ли критический путь

4. Мини-дискуссия

какая из пяти претензий к традиционному PM (слайд 12) больше всего актуальна для вашего учебного проекта — и почему